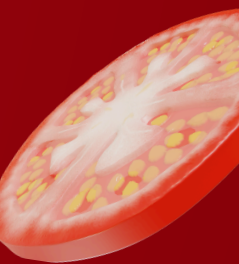
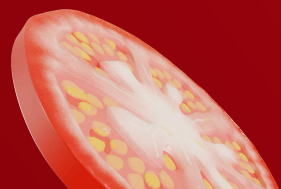


PIZZAHUT SALES ANALYSIS BY USING SQL



DESCRIPTION:

In this project, I analyzed Pizza Hut sales using SQL. I started by finding the total number of orders and the total revenue from pizza sales. I also identified the most expensive pizza, the most popular pizza size, and the top 5 pizzas ordered. I looked at when orders were placed to see the most common hours and checked how many pizzas were ordered on average per day. I also figured out which 3 pizza types made the most money and how much each pizza type contributed to the total revenue. Finally, I broke down the revenue for different pizza categories and found the top 3 pizzas in each category based on revenue



RETRIVE THE TOTAL NUMBER OF ORDERS PLACED ?

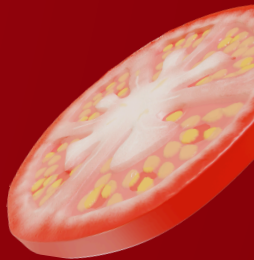
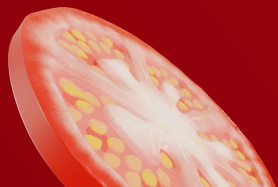
```
SELECT count(order_id) as total_orders FROM orders;
```

Result Grid	
	total_orders
▶	21350

CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES ?

```
SELECT
    SUM(order_details.quantity * pizzas.price) AS total_sales
FROM
    order_details
    JOIN
    pizzas ON pizzas.pizza_id = order_details.pizza_id
```

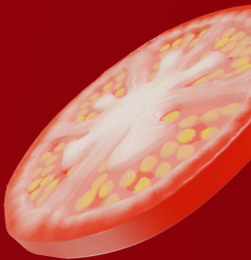
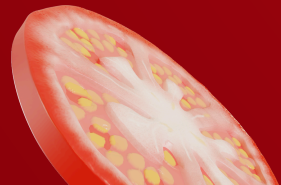
Result Grid	
	total_sales
▶	817860.04999999993



IDENTIFY THE HIGHEST - PRICED PIZZA ?

```
SELECT pizza_types.name, pizzas.price
FROM pizza_types JOIN pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
order by pizzas.price desc limit 1;
```

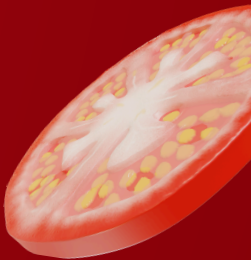
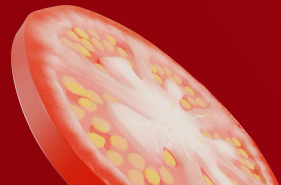
Result Grid			Filter Rows	
	name	price		
▶	The Greek Pizza	35.95		



IDENTIFY THE MOST PIZZA SIZE ORDERED ?

```
SELECT
  pizzas.size,
  COUNT(order_details.order_details_id) AS order_count
FROM
  pizzas
  JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```

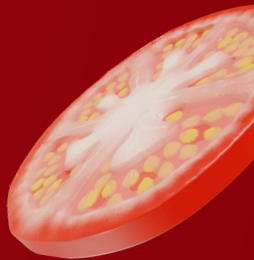
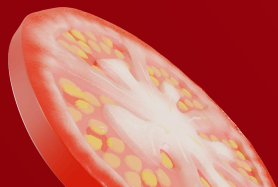
Result Grid			Filter
	size	order_count	
▶	L	18526	
	M	15385	
	S	14137	
	XL	544	14
	XXL	28	



LIST THE TOP MOST ORDERED PIZZAS TYPE ALONG WITH THEIR QUANTITIES

```
SELECT pizza_types.name,  
       SUM(order_details.quantity) AS total_quantity  
FROM pizza_types  
JOIN pizzas  
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_details  
  ON order_details.pizza_id = pizzas.pizza_id  
GROUP BY pizza_types.name  
ORDER BY total_quantity DESC  
LIMIT 5;
```

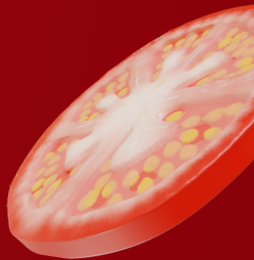
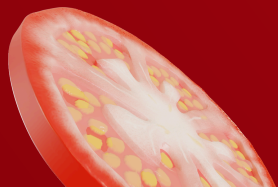
	name	total_quantity
▶	The Classic Deluxe Pizza	2453
	The Barbecue Chicken Pizza	2432
	The Hawaiian Pizza	2422
	The Pepperoni Pizza	2418
	The Thai Chicken Pizza	2371



JOIN THE NECESSARY TABLES TO FIND THE TOTAL QUANTITY OF EACH PIZZA CATEGORY ORDERED ?

```
SELECT pizza_types.category,  
sum(order_details.quantity) as quantity  
from pizza_types join pizzas  
on pizza_types.pizza_type_id = pizzas.pizza_type_id  
join order_details  
on order_details.pizza_id = pizzas.pizza_id  
group by pizza_types.category order by quantity desc;
```

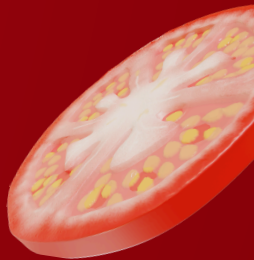
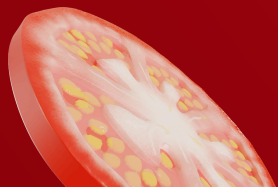
Result Grid		
	category	quantity
▶	Classic	14888
	Supreme	11987
	Veggie	11649
	Chicken	11050



DETERMINE THE DISTRIBUTION OF ORDERS BY HOUR OF THE DAY.

```
SELECT
  HOUR(order_time) AS hour, COUNT(order_id) AS order_count
FROM
  orders
GROUP BY HOUR(order_time);
```

Result Grid				Filter
	hour	order_count		
▶	11	1231		
	12	2520		
	13	2455		
	14	1472		
	15	1468		

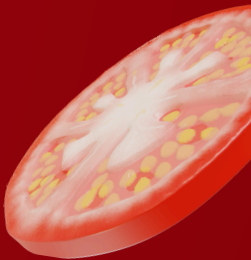
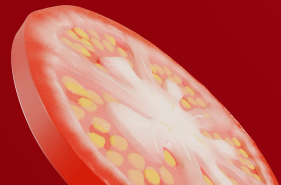


JOIN RELEVANT TABLES TO FIND THE CATEGORY-WISE DISTRIBUTION OF PIZZAS.

```
SELECT category, count(name) from pizza_types  
group by category;
```

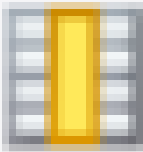
Result Grid |   Filter Rows

	category	count(name)
▶	Chicken	6
	Classic	8
	Supreme	9
	Veggie	9



GROUP THE ORDERS BY DATE AND CALCULATE THE AVERAGE NUMBER OF PIZZAS ORDERED PER DAY.

```
SELECT
  ROUND(AVG(quantity), 0) as avg_pizza_ordered_per_day
FROM
  (SELECT
    orders.order_date, SUM(order_details.quantity) AS quantity
  FROM
    orders
  JOIN order_details ON orders.order_id = order_details.order_id
  GROUP BY orders.order_date) AS order_quantity;
```

Result Grid   Filter Rows	
	avg_pizza_ordered_per_day
	138

DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE.

```
SELECT
  pizza_types.name,
  SUM(order_details.quantity * pizzas.price) AS revenue
FROM
  pizza_types
  JOIN
  pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
  JOIN
  order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Result Grid		Filter Rows:
name	revenue	
The Thai Chicken Pizza	43434.25	
The Barbecue Chicken Pizza	42768	
The California Chicken Pizza	41409.5	

CALCULATE THE PERCENTAGE CONTRIBUTION OF EACH PIZZA TYPE TO TOTAL REVENUE.

```
SELECT
  pizza_types.category,
  (SUM(order_details.quantity * pizzas.price) / (SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),2) AS total_sales
FROM
  order_details
  JOIN
  pizzas ON pizzas.pizza_id = order_details.pizza_id) ) * 100 as revenue
FROM
  pizza_types
  JOIN
  pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN
  order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue DESC;
```

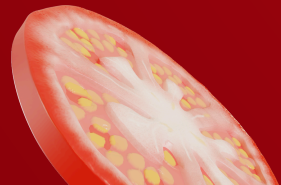
category	revenue
Classic	26.905960255666967
Supreme	25.45631126009862
Chicken	23.955137556847287
Veggie	23.682590927384577



ANALYZE THE CUMULATIVE REVENUE GENERATED OVER TIME ?

```
SELECT
  sales.order_date,
  SUM(sales.revenue) OVER (ORDER BY sales.order_date) AS
cum_revenue
FROM
  (SELECT
    orders.order_date,
    SUM(order_details.quantity * pizzas.price) AS revenue
  FROM
    order_details
  JOIN
    pizzas ON order_details.pizza_id = pizzas.pizza_id
  JOIN
    orders ON orders.order_id = order_details.order_id
  GROUP BY
    orders.order_date) AS sales;
```

Result Grid		Filter Rows:
order_date	cum_revenue	
2015-01-01	2713.8500000000000004	
2015-01-02	5445.75	
2015-01-03	8108.15	
2015-01-04	9863.6	
2015-01-05	11929.55	

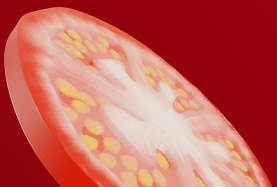


DETERMINE THE TOP 3 MOST ORDERED PIZZA TYPES BASED ON REVENUE FOR EACH PIZZA CATEGORY.

```
select name, revenue from

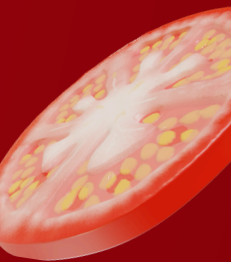
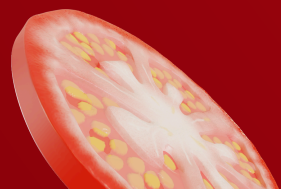
(SELECT
  category,
  name,
  revenue,
  RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
FROM
  (SELECT
    pizza_types.category,
    pizza_types.name,
    SUM(order_details.quantity * pizzas.price) AS revenue
  FROM
    pizza_types
  JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
  GROUP BY
    pizza_types.category, pizza_types.name) AS a) as b
where rn <= 3;
```

name	revenue
The Five Cheese Pizza	26066.5
The Mexicana Pizza	26780.75
The Pepperoni Pizza	30161.75
The Sicilian Pizza	30940.5
The Four Cheese Pizza	32265.700000000065



CONCLUSION

In conclusion, this analysis helped identify key sales trends and popular pizza choices at Pizza Hut. The insights can support better decision-making for inventory, marketing, and revenue growth.



THANK YOU!

